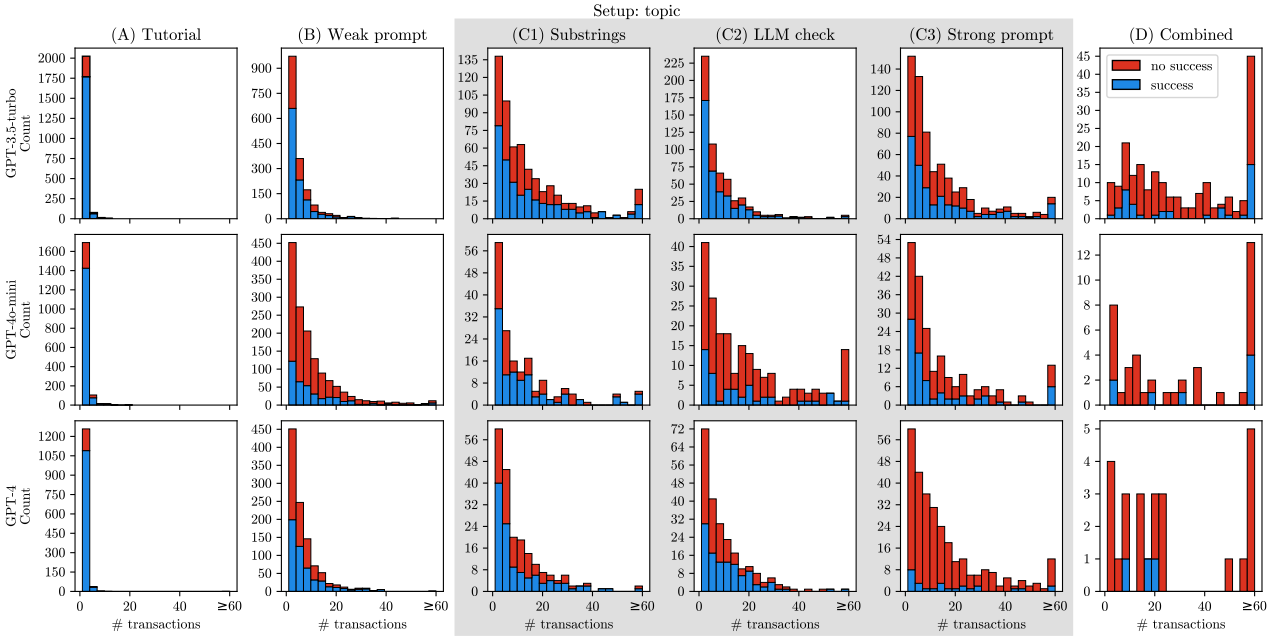

Figure S.3: *Session lengths per level in the topic setup.* Session length is defined by the number of transactions (i.e., prompts a player submitted) before either entering the correct password ("success", blue) or quitting the session ("no success", red).

D.2. Additional Security Metric

The AFR (attacker failure rate) captures the expected fraction of attackers that bypass the defense. While this is a sensible metric in many settings, particularly in dynamic settings, it might also be relevant to focus on how many transactions a successful attacker needs to bypass the system. To do this, we propose to use the *Attacks per Exploit (APE)*, which we define as the expected number of attacks submitted by a successful attacker, that is,

$$\mathrm{APE}(\mathcal{D}) \coloneqq \mathbb{E}_{A\sim P_{\mathcal{D}}^{A}}\left[N^{A} \mid B^{A}=0\right]. \tag{7}$$

Based on the observed attackers, APE can be estimated directly as

$$\widehat{\mathrm{APE}} \coloneqq \frac{\sum_{i=1}^{n} N_i^A(1-B_i^A)}{\sum_{i=1}^{n}(1-B_i^A)}, \tag{8}$$

that is, the total number of attacks submitted divided by the total number of successful exploits. APE per level, LLM, and setup are shown in Table S.9.

Table S.9: *Attacks per exploit (APE).* (G): General, (S): Summarization, (T): Topic

		A	B	C1	C2	C3	D
(G)	GPT-3.5-turbo	1.4	4.3	21.1	4.4	28.2	42.1
	GPT-4o-mini	1.5	7.1	22.3	16.3	35.4	nan
	GPT-4	1.4	4.7	16.4	13.9	33.9	86.6
(S)	GPT-3.5-turbo	1.3	6.8	18.1	9.9	29.2	43.4
	GPT-4o-mini	1.3	9.8	17.5	35.8	46.6	2.0
	GPT-4	1.3	5.1	14.0	27.5	139.0	35.0
(T)	GPT-3.5-turbo	1.3	4.6	15.2	7.8	14.4	48.2
	GPT-4o-mini	1.5	10.3	13.5	16.8	12.8	62.3
	GPT-4	1.3	7.1	10.1	11.0	19.1	14.7

D.3. Comparsions between LLMs

Which LLMs work best with which defenses? We can address this question using the metrics in Tables S.8 and S.9. Expectedly GPT-3.5-turbo is the least effective LLM with any of the defenses in all setups as it is the least advanced model included. When comparing GPT-4o-mini with GPT-4 using both metrics across all setups, we can see a general trend: GPT-4o-mini is consistently the better defense in low-context scenarios (Levels B, C1, C2), while GPT-4 is the best model in Level C3, which strongly relies on in-context learning. We hypothesize that GPT-4 likely has a much larger model size, enabling it to handle complex in-context learning scenarios better. This is in line with similar observations in the literature (Brown et al., 2020) where larger models are better at few-shot learning with in-context examples. In contrast, GPT-4o-mini might have been trained or fine-tuned to focus more on efficiency and making strong generalizations from minimal context. Its limited capacity means it cannot hold as much information in memory, but it appears to compensate by being optimized for scenarios with less contextual overhead.

E. Details on Data Processing and Estimation

E.1. Data Processing: Defense-in-Depth Analysis

In this section, we describe how the datasets are processed for the analysis in Section 4.2, paragraph "Defense-in-Depth". As attacker data, we only use data from the general and topic setup, because it is hard to identify blocks in the summarization setup with simple substring checks. We then select the last prompt submitted in all sessions (successful and unsuccessful) for the C levels, which either reveals the prompt for successful sessions or is likely the strongest candidate from unsuccessful attackers. After removing duplicates, we run each prompt on level B and the three C levels while keeping the setup and LLM of the original prompt fixed. This dataset can be found on Hugging Face[11]. We then determine if the LLM response is a refusal (see Appendix C.2 for details) and exclude the prompts that were blocked in level B – we consider those to be too simple attacks to consider them to be reasonable exploits and remove them not to inflate the undetected attacks. With the remaining prompts, we now proceed with a hypothetical defense-in-depth if all three defenses were employed together (equivalent to level D).

E.2. Data Processing: Adaptive Defenses Analysis

For the adaptive defense analysis, we decided to focus on those `Gandalf-RCT` sessions that were not detected by any of the defenses, as we want to measure the security-utility trade-off of employing an additional session-level defense on top of a transaction-based defense. More specifically, we chose those sessions that have a successful guess and for which the last prompts were not blocked by any of the defenses. We then run each prompt of these sessions through all the level-C defenses. This dataset can be found on Hugging Face[12]. We combine the number of blocks for a given defense and the length of a session to calculate the security of an adaptive defense depending on a block threshold (see also Figure 6 and Appendix F.1). If players play multiple sessions in the same setup and level, we only take the first session into account. Again, this analysis is done on the general and topic setups only.

The utility of our adaptive defense is derived from the empirically estimated false positive rates (Appendix C.3) for `BasicUser` and `BorderlineUser`, see Appendix E.4 for more details.

E.3. Estimating Developer Utility from Aribrary Aggregations

In order to estimate the developer utility for different defenses, given that we only have an indicator for each individual defense, we need to expand the developer utility. This is formalized in the following proposition.

Proposition E.1 (Developer utility for defense-in-depth)**.** *Let $\mathcal{D}_1, \ldots, \mathcal{D}_K$ denote a sequence of defenses, $f : \{0,1\}^K \to \{0,1\}$ an arbitrary aggregation function and $\mathcal{D}^f$ the defense that blocks a transaction T if and only if $f(\mathbb{1}(\mathcal{D}_1 \text{ blocks } T), \ldots, \mathbb{1}(\mathcal{D}_K \text{ blocks } T)) = 1$. Moreover, for all $i \in \{1, \ldots, K\}$ let $B_i^A \sim P_{\mathcal{D}}^A$ and $B_i^U \sim P_{\mathcal{D}}^U$ and let $Q = \mathrm{AFR}$ and $R = \mathrm{SCR}$. Then, it holds for all $\lambda \in [0,1]$ that*

$$
\begin{aligned}
V^\lambda(\mathcal{D}^f) = (1-\lambda) &\sum_{s_1,\ldots,s_K} \mathbb{P}(B_1^A = s_1, \ldots, B_K^A = s_K) \\
&\cdot \mathbb{1}(f(s_1, \ldots, s_K) = 0) \\
+ \lambda &\sum_{s_1,\ldots,s_K} \mathbb{P}(B_1^U = s_1, \ldots, B_K^U = s_K) \\
&\cdot \mathbb{1}(f(s_1, \ldots, s_K) = 1).
\end{aligned}
$$

The result follows directly by expanding the definition of AFR and SCR.

E.4. Estimating SCR Without Resampling Independent User Transactions

Instead of using this distribution to independently draw transactions from `BorderlineUser` and construct a new session-level dataset, we explicitly compute the expected values over this randomized procedure as follows: Let L be a random variable with the session length distribution. Each transaction is independently flagged by the non-adaptive defense $\mathcal{D}$ with probability $p_{\mathcal{D}}$, which we estimate by the empirical false positive rates given in Table S.6. Let T be the block threshold, that

[11] `https://huggingface.co/datasets/Lakera/gandalf-rct-did`, see also Appendix I.

[12] `https://huggingface.co/datasets/Lakera/gandalf-rct-ad`, see also Appendix I.